INFORMATION TECHNOLOGY UNIVERSITY

SE411L / MD442L: Full Stack Web & Mobile Development | Fall 2026

Lab 3 — Quran Index

Components

One component is new this week: **`SectionList`**. Everything else the lab needs (`View`, `Text`, `Image`, `Pressable`, `FlatList`, `useState`) was defined in Lab 01 and Lab 02, and JSON and string formatting have their own guide, `JSON-AND-STRINGS.md`.

The example below is deliberately **not** the Quran screen: other data, other names, other `testID`s. Read the pattern here, then write the lab's version yourself.

Components

`SectionList` — a list with a header over each group *(new in this lab)*

A `FlatList` takes **one flat array** and draws a row per item. A `SectionList` takes an **array of sections**, draws a header over each one and the rows under it, and still only renders what is near the screen. It is the component for data that already comes in groups: a menu by course, contacts by first letter, the surahs under each juz.

The shape it wants

Every section is an object with a `data` array. Anything else you put on the section (`title` here) is yours, and comes back to you in `renderSectionHeader`.

```
const sections = [
  { title: 'Drinks', data: [{ id: 'd1', name: 'Chai' }, { id: 'd2', name: 'Lassi' }] },
  { title: 'Snacks', data: [{ id: 's1', name: 'Samosa' }] },
];
```

Props

prop	what it does
`sections`	the array of `{ title, data }` objects
`renderItem`	called with `{ item, index, section }`, returns one row
`renderSectionHeader`	called with `{ section }`, returns the header over that group
`keyExtractor`	returns a **string** id, unique across the **whole** list, not just its section
`stickySectionHeaders Enabled`	the header stays pinned at the top while its rows scroll past
`ItemSeparatorComponent`	drawn between two rows of the same section
`contentContainerStyle`	padding *inside* the scrolling content, where a list's inset belongs
`ListEmptyComponent`	drawn when there are no sections at all
`testID`	on the list itself

A basic example: a JSON object held in state

`src/data/canteen.json`, one object with the groups inside it:

```
{
  "updated": "2026-08-01",
  "groups": [
    { "group": "Drinks", "items": [{ "id": "d1", "name": "Chai", "price": 60 },
                                   { "id": "d2", "name": "Lassi", "price": 120 }] },
    { "group": "Snacks", "items": [{ "id": "s1", "name": "Samosa", "price": 40 }] }
  ]
}
```

The screen imports that object, **saves it in state**, and builds the sections from it:

```
import React, { useState } from 'react';
import { SectionList, View, Text, StyleSheet } from 'react-native';

import canteen from '../data/canteen.json';

export default function CanteenScreen() {
  // The whole JSON object, held by the screen. useState reads `canteen` once,
  // on the first render, and `menu` is what the screen draws from then on.
  const [menu, setMenu] = useState(canteen);

  // JSON shape -> SectionList shape. One section per group, `data` per section.
  const sections = menu.groups.map(group => ({
    title: group.group,
    data: group.items,
  }));

  return (
    <SectionList
      testID="canteen-list"
      sections={sections}
      keyExtractor={item => item.id}
      stickySectionHeadersEnabled
      renderSectionHeader={({ section }) => (
        <Text testID={`canteen-header-${section.title}`} style={styles.header}>
          {section.title}
        </Text>
      )}
      renderItem={({ item }) => (
        <View testID={`canteen-row-${item.id}`} style={styles.row}>
          <Text style={styles.name}>{item.name}</Text>
          <Text style={styles.price}>{`Rs ${item.price}`}</Text>
        </View>
      )}
    />
  );
}
```

```
const styles = StyleSheet.create({
  header: { backgroundColor: '#E1E6F3', paddingHorizontal: 16, paddingVertical: 8,
            fontSize: 13, fontWeight: '700', color: '#1B2550' },
  row: { flexDirection: 'row', justifyContent: 'space-between',
         paddingHorizontal: 16, paddingVertical: 12, backgroundColor: '#FFFFFF' },
  name: { fontSize: 16, color: '#1B2550' },
  price: { fontSize: 16, color: '#5E6886' },
});
```

Three things that example is showing you:

- **The JSON is state, not a global.** `useState(canteen)` takes the imported object as the first value and hands back `menu` plus `setMenu`. The screen reads `menu`, so the day the data comes from storage or a server instead of a file, only that one line changes.
- **`setMenu` replaces, it never edits.** `menu.groups.push(...)` changes the object in place and React sees no new value, so nothing redraws. Build a new object and set it: `setMenu({ ...menu, groups: nextGroups })`.
- **The sections are built from the state, every render.** The JSON keeps its own shape and the screen makes the shape `SectionList` wants. Nothing is copied into a second source of truth.

`renderItem` also receives its section

`index` is the item's position **within its own section**, and `section` is the section it belongs to, so a row can show both:

```
renderItem={({ item, index, section }) => (
  <Text>{`${section.title} ${index + 1}: ${item.name}`}</Text>
)}
```

Keys are unique across the whole list

`keyExtractor` runs over every item of every section, so two rows in **different** sections may not return the same string. When the same item legitimately appears in two sections, make the key from the pair, not from the item alone:

```
keyExtractor={item => item.id}                          // fine while each item appears once
keyExtractor={(item, index) => `${item.id}-${index}`} // Wrong: index keys break on
    ↪ reorder
```

Give each item its own id when you build the sections, and `keyExtractor` stays one line. The lab's rows have this problem for real: work out what makes one of its rows unique.

Common mistakes

Wrong	Right
`data={sections}`	`sections={sections}` — the prop has a different name from `FlatList`'s
a section without `data`	every section object must have a `data` array, even an empty one
`renderSectionHeader={({ title }) => ...}`	the argument is `{ section }`: `({ section }) => section.title`
a header returned as a bare string	it must be an element: `<Text>{section.title}</Text>`
`keyExtractor` returning a number	it must return a **string**: `String(item.id)`
`sections.map(...)` inside a `ScrollView`	that builds every row on every render, which is what a list component exists to avoid

References

- `SectionList` — https://reactnative.dev/docs/sectionlist
- `useState` — https://react.dev/reference/react/useState
- JSON and string formatting in this starter — `JSON-AND-STRINGS.md`